

Project Book

Radar de Tendências Tecnológicas — SENAI Futuro — IA

Project Book — Radar de Tendências Tecnológicas

Documento de referência único do projeto: visão, escopo, arquitetura, decisões, qualidade, avaliação crítica e roadmap. Escrito para quem avalia o projeto pela primeira vez ou herda sua manutenção sem contexto prévio. Última atualização: 2026-07-07.

Sumário

1. [Resumo executivo](#)
 2. [Problema e escopo](#)
 3. [Visão geral da solução](#)
 4. [Arquitetura](#)
 5. [Decisões técnicas de design](#)
 6. [Modelo de dados](#)
 7. [Qualidade e estratégia de testes](#)
 8. [Gestão do conhecimento e documentação](#)
 9. [Avaliação crítica](#)
 10. [Custos operacionais](#)
 11. [Roadmap — evoluções futuras](#)
 12. [Glossário](#)
 13. [Referências e apêndices](#)
-

1. Resumo executivo

O **Radar de Tendências Tecnológicas** é uma plataforma que recebe um tema tecnológico em texto livre (ex.: "Edge AI", "Robôs Humanoides para Indústria") e devolve, em minutos, um **painel executivo de 10 seções** fundamentado em evidências verificáveis coletadas na internet — cada afirmação rastreável a uma fonte citada, com grau de suporte calculado de forma determinística e auditável, nunca por autoavaliação do modelo de IA.

Desenvolvido para o desafio **SENAI Futuro — IA**, o projeto combina:

- **Coleta acadêmica determinística** (arXiv, OpenAlex) — sem LLM, sem custo de tokens.
- **Um Agente Coletor** no Azure AI Foundry, com a ferramenta Web Search e uma técnica de perguntas multi-perspectiva inspirada na pesquisa **STORM** (Stanford OVAL).
- **Um Agente Sintetizador**, que consolida o corpus em um painel estruturado.

- **Persistência em Cosmos DB** com histórico consultável e um modo offline dedicado à resiliência de demonstrações.

O sistema está em produção (`radar-tendencias-app.azurewebsites.net`), coberto por 50 testes automatizados, com uma prática de **avaliação crítica documentada continuamente** (`docs/critical-review.md`) que registra bugs reais, limitações e custos à medida que são descobertos — não retroativamente, na véspera da entrega.

2. Problema e escopo

2.1 O problema

Entender uma tendência tecnológica emergente exige cruzar fontes heterogêneas — literatura científica, análises de mercado, notícias, sinais de adoção industrial e patentes — e avaliar criticamente o quanto cada afirmação é, de fato, sustentada por evidência real, não apenas plausível. Fazer isso manualmente é lento; delegar inteiramente a um LLM sem rastreabilidade é arriscado (alucinação sem aviso).

2.2 O que o sistema faz (requisitos funcionais centrais)

Requisito	Resumo
FR-001	Aceita um tema em texto livre como entrada única
FR-002/003	Coleta evidências de ≥ 2 naturezas (acadêmica + mercado), com 4 perspectivas na coleta de mercado
FR-004	Consolida em painel de 10 seções fixas
FR-005/006	Grau de suporte por seção + toda afirmação rastreável ou marcada como inferência
FR-007	Deduplica evidências equivalentes antes de graduar
FR-009/010/011	Persiste, mantém histórico consultável, reabre sem nova coleta
FR-012/013	Falha de uma fonte não aborta a análise; timeout entrega parcial, nunca falha silenciosa
FR-015	Lista completa de evidências, filtrável por tipo

Lista completa em `specs/001-radar-tendencias/spec.md`.

2.3 Fora de escopo (decisão deliberada)

- Multiusuário e autenticação (usuário único no MVP).
- API dedicada de patentes (cobertas via sinais de busca web).
- Monitoramento contínuo agendado de temas (roadmap, não implementado).
- Acesso a texto completo pago/paywall — evidências baseadas em abstracts e cobertura pública.

2.4 Critérios de sucesso mensuráveis

Critério	Meta	Status
SC-001	Painel completo em ≤5 min	✓ validado (smoke test: 36,8-123,0s)
SC-002	100% das seções com referência ou inferência marcada	✓ (validação Pydantic garante)
SC-003	Histórico abre em <5s sem rede externa	✓ (point read por partition key)
SC-004	Degradação de 1 fonte não impede conclusão	✓ (testado com falha simulada)
SC-005	≥3 tipos de fonte distintos nos temas do desafio	✓ (smoke test: 3-4 tipos)
SC-006	Rastrear qualquer afirmação em ≤2 cliques	✓ (seção → expandir de referências)
SC-007	Demo completa executável em <10 min	✓

3. Visão geral da solução

Em uma frase: **Streamlit no Azure App Service orquestra um pipeline em Python que combina coleta acadêmica determinística com um Agente Coletor multi-perspectiva no Azure AI Foundry e um Agente Sintetizador, entregando um painel de 10 seções com grau de suporte calculado em código — persistido no Cosmos DB, com histórico e modo offline.**

Fluxo de uma análise (resumo)

14. Usuário informa o tema → guardrail de escopo classifica se é um tema tecnológico válido (barato, roda antes de qualquer coleta cara).
15. Documento Report(status=running) persistido imediatamente.
16. Coleta acadêmica (arXiv + OpenAlex) e coleta de mercado (Agente Coletor, 4 perguntas concorrentes) rodam **em paralelo**.
17. Evidências combinadas e deduplicadas.
18. Corpus deduplicado e numerado → Agente Sintetizador → JSON validado contra schema.
19. Grau de suporte calculado por seção, em código puro.
20. Report(status=completed|partial|failed) persistido; painel renderizado com gráficos de visão geral (Plotly).

Ver docs/architecture.md para o diagrama de componentes completo e o passo a passo técnico detalhado, e a explicação em linguagem simples do orquestrador e dos agentes no Q&A de arguição (docs/apresentacao/arguicao.md).

4. Arquitetura

4.1 Camadas (identificadas via análise automatizada do código-fonte)

Camada	Responsabilidade	Diretório
Apresentação (UI Streamlit)	Recebe o tema, dispara o pipeline, renderiza o painel e os gráficos	app/
Agentes e Coletores	Agente Coletor (Foundry), Agente Sintetizador, coletores acadêmicos determinísticos	src/radar/agents/, src/radar/collectors/
Síntese e Lógica de Domínio	Deduplicação e cálculo de grau de suporte — funções puras	src/radar/synthesis/
Persistência	Interface de repositório + implementações Cosmos DB e local	src/radar/storage/
Núcleo e Orquestração	Modelos Pydantic, configuração, orquestrador do pipeline	src/radar/models.py, config.py, orchestrator.py
Testes	Unitários e de integração, com	tests/

Camada	Responsabilidade	Diretório
	fixtures mockadas	
Infraestrutura e Operações	Scripts de provisionamento, smoke test, exportação	infra/
Documentação e Apresentação	Docs de arquitetura/crítica, material de arguição	docs/
Especificação (Spec-Kit)	Constituição, spec, plano, pesquisa, tarefas	specs/001-radar-tendencias/, .specify/

4.2 Padrões de design aplicados

- **Strategy:** `Collector` (interface abstrata em `collectors/base.py`) — arXiv e OpenAlex a implementam; um novo coletor se pluga sem alterar o orquestrador.
- **Repository + Factory:** `ReportRepository` (interface) com duas implementações (Cosmos DB / local); `storage/__init__.py` escolhe qual instanciar conforme `RADAR_OFFLINE`. Orquestrador e UI dependem só da interface.
- **Fail-open em componentes auxiliares:** o guardrail de escopo nunca bloqueia um tema legítimo por falha própria — princípio de resiliência aplicado deliberadamente (Princípio IV da constituição).

4.3 Infraestrutura Azure

Todos os serviços colocalizados em `brazilsouth` (App Service, Cosmos DB, Foundry) — decisão que elimina latência cross-region e nasceu de uma restrição real (eastus sem capacidade de Cosmos DB no momento do provisionamento). Detalhes de provisionamento em `infra/provision.md`; procedimentos operacionais em `docs/runbook.md`.

5. Decisões técnicas de design

Decisões completas em formato **Decision / Rationale / Alternatives** (equivalente a ADRs) em `specs/001-radar-tendencias/research.md` (R1-R11). Resumo das mais relevantes para avaliação:

#	Decisão	Por quê
R1	Web Search tool nativo (não Bing Grounding classic)	GPT-4.1/4o em deprecating nesta conta; recurso Bing

#	Decisão	Por quê
		Grounding inelegível na assinatura — achados reais de provisionamento, não preferência
R2	4 perguntas multi-perspectiva concorrentes , inspiradas em STORM (Stanford OVAL)	Sequencial estouraria o limite de 5 min (medido no spike: ~30s/pergunta)
R3	arXiv + OpenAlex via HTTP direto, sem framework de agente	Metadados estruturados, sem chave, sem fricção
R5	Documento agregado único por relatório no Cosmos	Leitura sempre conjunta (painel + evidências); elimina joins
R8	Grau de suporte calculado em código, nunca pelo LLM	Auditável, testável, defesa contra alucinação/autoavaliação enviesada
R9	Modo offline (RADAR_OFFLINE=1)	Resiliência de demo — Cosmos não ajuda se a rede do apresentador cair
R10	Freios de custo (rate limit diário, budget alert, Access Restrictions)	App público sem auth executando pipeline pago
R11	Guardrail de escopo antes do pipeline caro	Lacuna real identificada — qualquer tema disparava o pipeline inteiro sem checagem

A técnica multi-perspectiva (R2) é adaptada — não importada — da pesquisa publicada: Shao et al., *STORM*, NAACL 2024 ([arXiv:2402.14207](https://arxiv.org/abs/2402.14207)), Stanford OVAL. Apenas o princípio de perguntas fixas sob perspectivas diferentes foi reproduzido; simulação de conversa multi-agente, refinamento iterativo e o protocolo colaborativo do Co-STORM (Jiang et al., EMNLP 2024) **não** foram implementados — registrados como evolução futura, não superestimados na apresentação.

6. Modelo de dados

Modelos Pydantic (`src/radar/models.py`), persistidos como **um documento agregado por relatório** no Cosmos DB:

- ``Evidence``: título, tipo de fonte (`scientific/market/news/corporate/patent`),

origem, URL (obrigatória), data de publicação (quando disponível), snippet, idioma, perspectiva (só coleta de mercado), contagem de citações (só acadêmica).

- ``PanelSection``: uma das 10 seções fixas, conteúdo em markdown, ids de evidência citados, flag de inferência, nota de divergência, grau de suporte calculado.
- ``SupportGrade``: objeto calculado — contagem de evidências, contagem de tipos de fonte distintos, nível (high/medium/low/inference), flag de divergência.
- ``Report``: documento raiz — id, tema, slug (partition key), status, as 10 seções, o corpus completo de evidências, fontes degradadas, avisos, métricas.

Esquema completo com regras de validação em `specs/001-radar-tendencias/data-model.md` e o contrato formal (JSON Schema) que a saída do Sintetizador precisa obedecer em `specs/001-radar-tendencias/contracts/report-schema.json`.

7. Qualidade e estratégia de testes

- **50 testes automatizados** (pytest), cobrindo os módulos críticos por exigência da constituição (Princípio V): deduplicação, cálculo de grau de suporte, parsing e validação da saída do Sintetizador, coletores acadêmicos, guardrail de escopo, e o orquestrador completo (caminho feliz, degradação parcial/total, timeout, erros de API, tema fora de escopo).
 - Testes mockados (via respX) rodam sem custo e sem credenciais — adequados a CI.
 - `pytest -m live` roda um smoke test real contra Foundry/Cosmos de produção, reservado para validação antes de mudanças sensíveis (prompts, parsing de LLM, integrações externas) — é justamente onde os bugs reais mais graves só apareceram (ver §9).
 - Lint (ruff) limpo como gate de qualidade.

8. Gestão do conhecimento e documentação

A maturidade de documentação deste projeto vai além de um README: é uma pilha deliberada de artefatos, cada um com um público e propósito distintos.

Artefato	Papel
<code>README.md</code>	Ponto de entrada — visão geral, arquitetura em 1 parágrafo, quickstart
<code>docs/architecture.md</code>	Decisões técnicas com justificativa, diagrama de componentes
<code>docs/critical-review.md</code>	Autoavaliação crítica contínua — vieses, limitações, custos, incidentes reais datados

Artefato	Papel
<code>docs/runbook.md</code>	Operação: deploy, configuração, monitoramento, resposta a incidentes reais
<code>docs/project-book.md</code>	Este documento — referência única para avaliação/herança
Spec-Kit (<code>specs/001-radar-tendencias/</code>)	Especificação formal e rastreável: constitution → spec → clarify → plan → research → tasks
<code>docs/apresentacao/</code>	Guia de arguição, mapeamento aos critérios da banca, diagramas de apoio
<code>.understand-anything/knowledge-graph.json</code>	Grafo de conhecimento do código-fonte (ver 8.1)

8.1 Inovação: grafo de conhecimento como gestão de conhecimento do código

Diferente da documentação tradicional (que descreve a intenção e pode ficar desatualizada silenciosamente), este projeto mantém um **grafo de conhecimento extraído automaticamente do código-fonte real** via análise assistida por IA:

- **194 nós** (51 arquivos, 95 funções, 20 classes, 10 configurações, 18 documentos)
- **334 arestas** tipadas (imports, chamadas, contém, testado-por, documenta, etc.)
- **9 camadas arquiteturais** inferidas da estrutura real de dependências, não declaradas manualmente
- **15 passos de tour guiado**, navegando a jornada real do código (do ponto de entrada `app/Home.py` até a persistência), com "lições de linguagem" contextuais explicando padrões usados (`async/await`, `Pydantic`, `ABC/Strategy`, `DefaultAzureCredential`)

Diferencial: como o grafo é derivado do código e não de descrição manual, ele não sofre do problema clássico de documentação-que-mente — uma nova análise sempre reflete o estado real do repositório. É navegável via dashboard interativo (`http://127.0.0.1:5173` ao rodar `/understand-dashboard`), útil tanto para onboarding técnico quanto para a banca visualizar a estrutura sem precisar ler cada arquivo.

8.2 Práticas e normas seguidas

- **Spec-Driven Development** via Spec-Kit: nenhuma implementação começou sem `constitution.md` → `spec.md` → `plan.md` → `tasks.md` explícitos e versionados.
- **Decisões registradas em formato ADR** (Decision/Rationale/Alternatives) em `research.md` — nenhuma escolha técnica relevante ficou implícita.

- **Revisão crítica contínua**, não uma seção escrita na véspera: `critical-review.md` é atualizado a cada achado real, com data.
- **Versionamento semântico via git**, commits atômicos, sem force-push.
- **Constituição do projeto com princípios não-negociáveis** (I. Evidência Rastreável, IV. Resiliência de Demo, V. Qualidade Avaliável, VI. Avaliação Crítica Documentada, entre outros) que qualquer decisão de design precisa citar.

9. Avaliação crítica

Resumo — detalhes completos e datados em `docs/critical-review.md`.

9.1 Vieses conhecidos

- **De fonte**: consultorias pagas só entram pelo material público que divulgam; arXiv envia para preprints de CS/física (mitigado por OpenAlex).
- **De idioma**: corpus predominantemente em inglês.
- **De modelo**: Sintetizador pode privilegiar narrativas dominantes do treinamento — mitigado estruturalmente (só afirma com citação; grau nunca autoavaliado).
- **De recorte do buscador**: ranking do provedor por trás do Web Search decide o que o Coletor vê — mitigado parcialmente pelas 4 perspectivas.
- **De cobertura de metadado** (achado real 2026-07-07): evidências científicas quase sempre têm data de publicação estruturada; evidências de mercado/notícia via Web Search raramente têm — em temas com forte presença de mercado, só ~24% das evidências têm data. Os gráficos do painel tratam isso honestamente (computam "Publicações por ano" só sobre o subconjunto datado, com aviso quando insuficiente).

9.2 Limitações conhecidas

Patentes só por sinais de busca web (sem API dedicada); usuário único sem auth; análise síncrona in-process no Streamlit (não sobrevive a restart do App Service); capacidade de deployment moderada (pode gargalar sob uso concorrente real).

9.3 Incidentes reais e maturidade de resposta

Três classes de bug **só apareceram contra APIs reais**, nunca em testes mockados — achado que reforça a importância do smoke test `pytest -m live`:

- 21. Registro malformatado do OpenAlex (`source=null`) derrubava a coleta inteira.
- 22. arXiv migrou para HTTPS — redirect não seguido pelo cliente HTTP.
- 23. Erro de API (`RateLimitError`) não tratado na síntese travava o processo.

Mais um bug de produção real (não de código): `COSMOS_KEY` gravado vazio por uma falha de rede silenciosa durante o provisionamento — corrigido e documentado com a lição operacional (verificar comprimento do segredo, não apenas presença do nome). Todos os 4 incidentes têm resposta documentada em `docs/runbook.md` §6.

10. Custos operacionais

Item	Estimativa
Fixo mensal (App Service B1 + Cosmos DB serverless)	~US\$14-15/mês
Por análise (tokens do Foundry + Web Search tool)	~US\$0,20-0,50 (a confirmar via Azure Cost Management)

Freios de abuso (app público sem auth): limite diário de 10 análises, Access Restrictions por IP, budget alert de US\$30/mês, capacidade de token moderada no deployment. Detalhes em `docs/critical-review.md` §Custos e `docs/runbook.md` §7.

11. Roadmap — evoluções futuras

Priorizadas por valor/esforço, nenhuma implementada nesta fase (deliberadamente, para mostrar fronteiras conscientes do sistema, não descobertas de surpresa):

- 24. Adoção da metodologia **Co-STORM** (Jiang et al., EMNLP 2024) para melhorar coleta e síntese — discurso colaborativo entre agentes com papéis distintos e mapa mental dinâmico compartilhado: perguntas de busca se refinam a partir do que já foi encontrado (em vez das 4 perspectivas fixas de hoje) e o corpus chega pré-estruturado por tópico ao Sintetizador; também habilita um modo de exploração guiada, com o usuário participando do refinamento das perguntas.
- 25. API dedicada de patentes (EPO OPS, cobertura mundial incl. Brasil, gratuita).
- 26. Autenticação multiusuário (Easy Auth/Entra ID) + Key Vault/RBAC no Cosmos.
- 27. Monitoramento contínuo agendado de temas (Azure Functions timer) com diff entre execuções.
- 28. Observabilidade de custo por consulta (Application Insights + `Report.metrics`).
- 29. Avaliação de qualidade de evidência por fonte (ranking de confiabilidade).
- 30. Inferência de ano de publicação pelo Agente Coletor quando a página não expõe metadata estruturada (aumenta a cobertura do gráfico "Publicações por ano").

31. Agente de IA para refinamento semântico de query dos coletores acadêmicos — complementa a heurística determinística atual (`industrial_scope.py`) com nuance de domínio que uma lista fixa de termos não alcança.

12. Glossário

Termo	Significado
Agente Coletor	Agente de IA no Azure AI Foundry que busca evidências de mercado via Web Search, sob 4 perspectivas
Agente Sintetizador	Agente de IA que consolida o corpus de evidências em um painel de 10 seções
Grau de suporte	Classificação determinística (alto/médio/baixo/inferência) de quão bem uma seção é sustentada por evidências
STORM	Técnica de pesquisa (Stanford OVAL) de geração de perguntas multi-perspectiva; adaptada, não importada, neste projeto
Guardrail de escopo	Checagem barata que rejeita temas fora do domínio industrial/tecnológico antes do pipeline caro
Modo offline	<code>RADAR_OFFLINE=1</code> — troca Cosmos DB por arquivos JSON locais, para resiliência de demo
Grafo de conhecimento	Representação em nós/arestas do código-fonte real, extraída via IA, navegável por tour guiado

13. Referências e apêndices

- Especificação funcional completa: `specs/001-radar-tendencias/spec.md`
- Decisões técnicas (ADR-style): `specs/001-radar-tendencias/research.md`
- Modelo de dados: `specs/001-radar-tendencias/data-model.md`
- Constituição do projeto: `.specify/memory/constitution.md`
- Runbook operacional: `docs/runbook.md`
- Provisionamento de infraestrutura: `infra/provision.md`
- Avaliação crítica (documento vivo): `docs/critical-review.md`
- Guia de arguição e critérios de avaliação: `docs/apresentacao/arguicao.md`, `docs/apresentacao/criterios-avaliacao.md`

- Diagramas de arquitetura: docs/apresentacao/criterios/*.drawio.png
- Grafo de conhecimento navegável: .understand-anything/knowledge-graph.json

Referências acadêmicas citadas

- Shao, Y., Jiang, Y., Kanell, T. A., Xu, P., Khattab, O., & Lam, M. S. (2024). *Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models*. NAACL 2024. [arXiv:2402.14207](https://arxiv.org/abs/2402.14207)
- Jiang, Y., Shao, Y., Ma, D., Semnani, S., & Lam, M. (2024). *Into the Unknown Unknowns: Engaged Human Learning through Participation in Language Model Agent Conversations*. EMNLP 2024. [arXiv:2408.15232](https://arxiv.org/abs/2408.15232)